TASK SCHEDULER SYSTEM

Real Time Research Project — Review II

PRESENTED BY

CH. VAISHNAVI
247R1A05T9

P. RAAJITHA
247R1A05X8

P. HEMANTH REDDY
247R1A05X5

Abstract — Introduction & Overview of the Project

What is the Task Scheduler System?

The Task Scheduler System helps users manage tasks easily. It works for personal, academic, and work settings.

Users can set priorities, schedule tasks, and track progress. Reminders and notifications help meet deadlines on time.

It supports repeating tasks and connects with calendars. This gives users a clear view of all their work in one place.

3 User Roles

7 Core Modules

24/7 Automation

100% Encrypted

Existing System

EXISTING SYSTEM

Current task management systems vary widely in functionality, ranging from simple to complex applications. Most tools only offer to-do lists and basic reminders. They are not enough for complex needs.

Simple apps let users write tasks but lack smart scheduling and tool integrations.

No dynamic adaptation to changing workloads; users must manually determine task priority and reschedule.

Existing tools operate in isolation — no integration with calendars, email, or project management platforms.

Complex interfaces lead users to abandon digital tools in favor of handwritten lists.

Team environments lack real-time collaborative features, hindering coordination and project tracking.

Disadvantages of Existing System

01	**Limited Functionality**	Most tools only have basic features. They are not useful for users with complex tasks.
02	**Inflexibility**	Users must manually reschedule tasks when priorities change. This wastes time and causes missed deadlines.
03	**Poor Integration**	Most tools do not connect with calendars or other apps. This makes it hard to see all tasks in one place.
04	**User Interface Challenges**	Complex interfaces confuse users. Many people give up and use pen and paper instead.
05	**Insufficient Collaboration**	Teams cannot share tasks or track each other's work. This slows down team projects.

Proposed System

PROPOSED SYSTEM

Our system fixes the problems of current tools. It makes scheduling, prioritizing, and tracking tasks simple and fast.

- **Advanced Task Management:** Add tasks and sort them by urgency, importance, or due date.
- **Intelligent Scheduling Algorithms:** Smart algorithms suggest the best time to do each task.
- **Real-Time Collaboration:** Team members can share tasks and track work together in real time.
- **Integration with Productivity Tools:** Connects with calendars and email for a complete view of all work.
- **User-Friendly Interface:** Easy drag-and-drop layout with color-coded priorities and a custom dashboard.

Advantages of Proposed System

01	**Enhanced Productivity**	Users focus on important tasks first. This saves time and boosts productivity.
02	**Intelligent Task Management**	Smart algorithms schedule tasks at the best times. This reduces delays and procrastination.
03	**Real-Time Collaboration**	Teams share tasks and track progress together. Communication becomes easier and projects run smoothly.
04	**Seamless Integration**	All tools and calendars connect in one place. No deadline is missed.
05	**User-Friendly Interface**	The simple design is easy to use. Drag-and-drop and dashboards make the experience smooth.

Hardware & Software Requirements

HARDWARE REQUIREMENTS

Processor

Pentium IV 2.4 GHz

Hard Disk

40 GB

RAM

512 MB

SOFTWARE REQUIREMENTS

Operating System

Windows

Coding Language

Python 3.7.2

Database

MySQL

Novelty of the Project

01

Encrypted User Authentication
Passwords are encrypted. Only the right user can log in.

02

Real-Time Email Notifications
Daily email alerts remind users of pending tasks at a set time.

03

Intelligent Task Prioritization
Tasks are sorted by due date so users always know what to do next.

04

Full CRUD Task Lifecycle
Users can create, view, edit, delete, and complete tasks easily.

05

Python + MySQL Web Stack
Built with Python and MySQL. Works on any platform and is easy to deploy.

06

AI & NLP Ready Architecture
Ready for AI scheduling, voice commands, and smart device support in the future.

System Architecture

3-Tier Web Architecture

HTTP ⇕

PRESENTATION LAYER

Web Browser (HTML / CSS / JavaScript)
User Interface • Dashboard • Task Views • Notification Panel

▼

SQL ⇕

APPLICATION LAYER

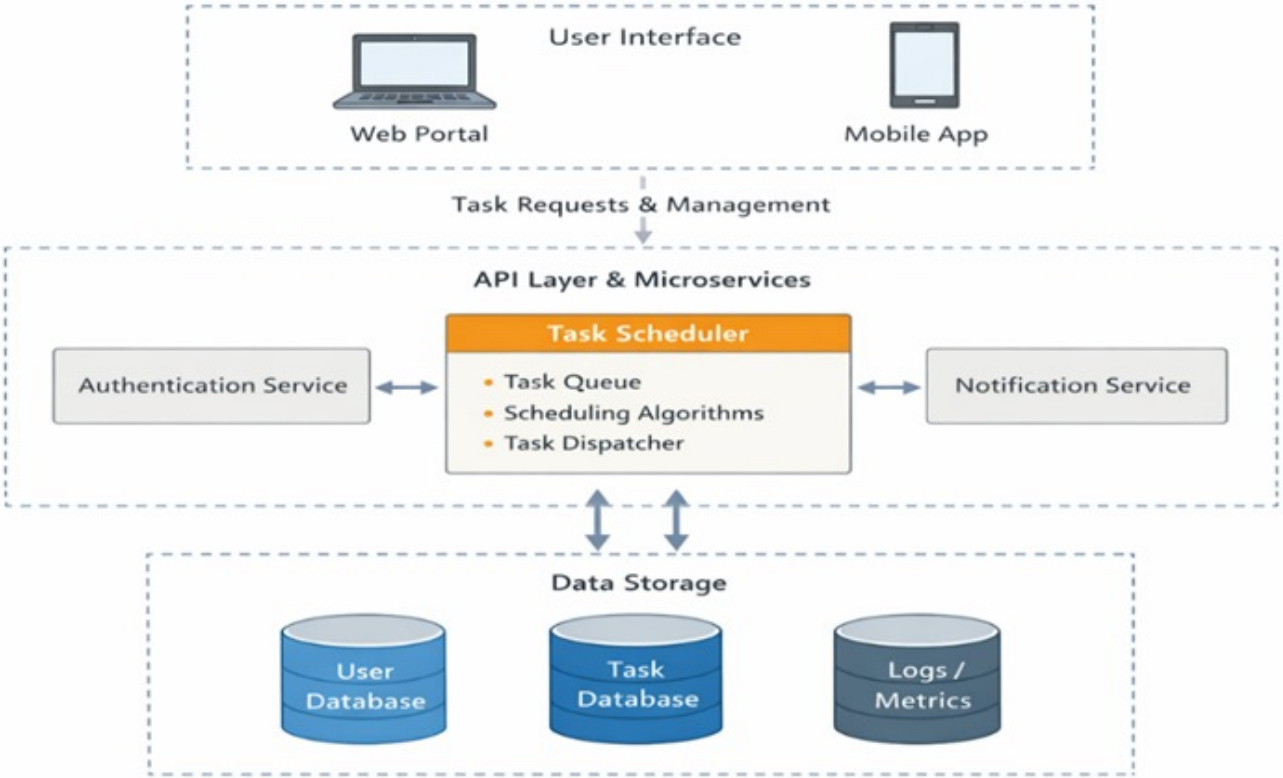
Python Server (run.bat → http://127.0.0.1:8000)
Task Logic • Scheduler Engine • SMTP Notification • Auth Module

▼

DATA LAYER

MySQL Database
Users • Tasks • Encrypted Passwords • Notification Settings

Task Scheduling System Architecture



System Modules

1

New User Sign Up
User registers with encrypted password — secure account creation using hashing algorithm.

2

User Login
Registered users authenticate and access the full task management dashboard securely.

3

Reset Password
Users can update their account password at any time for security maintenance.

4

Create Task
Add task with title, description, and due-date — reminder is automatically scheduled.

5

View / Edit / Delete Task
View all pending tasks and edit or delete individual entries as priorities shift.

6

Mark Complete
Mark a task as done once finished — removes it from pending list and updates status.

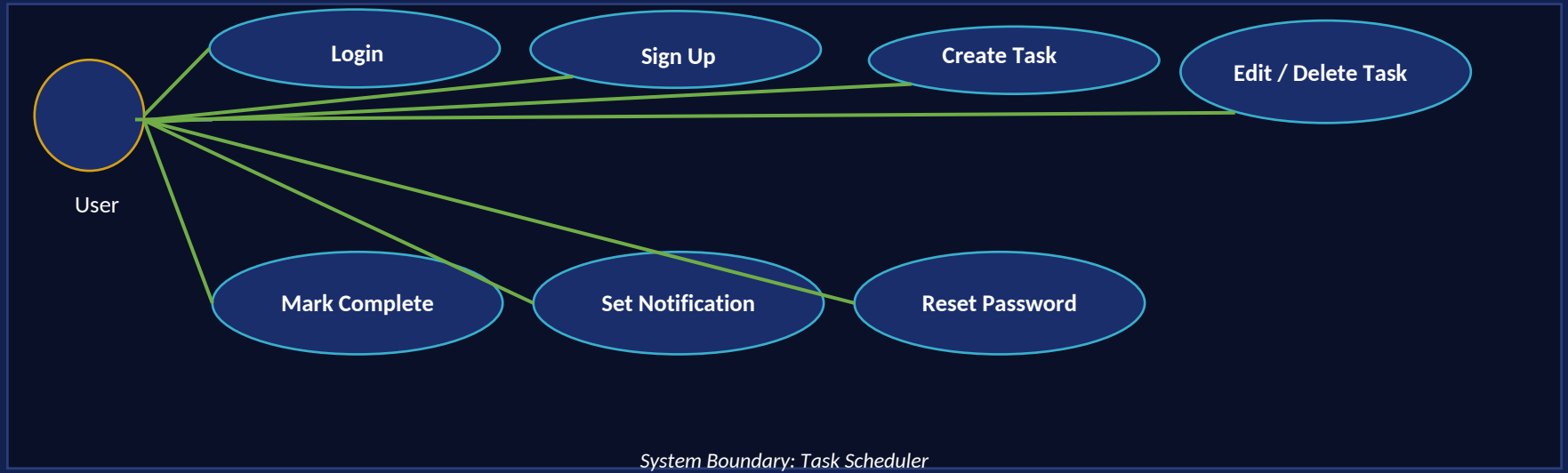
7

Set Notification Time — Configure a daily alert time; system automatically emails pending task alerts to the uday.

UML Diagrams — Use Case Diagram

USE CASE DIAGRAM

A Use Case Diagram shows what actions a user can perform in the system. It maps each feature to the user who uses it.



UML Diagrams — Class Diagram

CLASS DIAGRAM

A Class Diagram shows the data and actions for each part of the system. Classes are connected by relationships like ownership or inheritance.

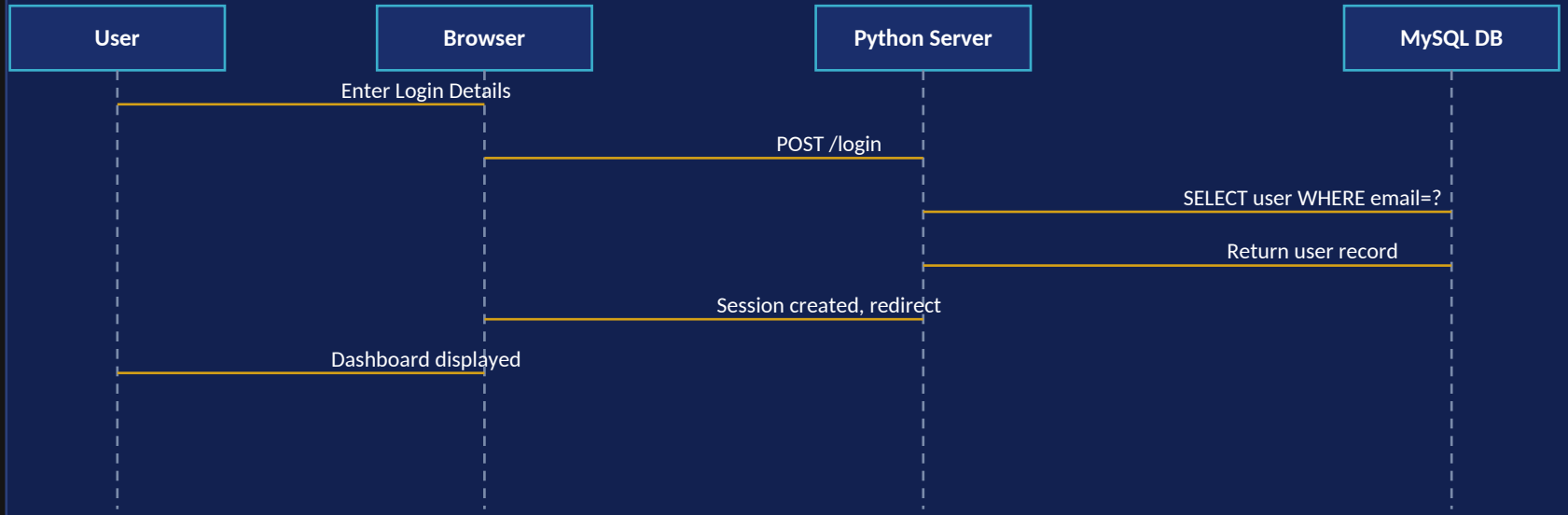
User	Task	Notification	Scheduler
- userId: int - name: String - email: String - password: String (encrypted)	- taskId: int - title: String - description: String - dueDate: DateTime	- notifId: int - userId: int - alertTime: Time	- schedId: int - tasks: List<Task>
+ login() + register() + resetPassword()	+ create() + edit() + delete() + markComplete()	+ setTime() + sendEmail()	+ run() + prioritize() + checkDue()

User → has many → Task | User → has → Notification | Scheduler → manages → Task

UML Diagrams — Sequence Diagram

SEQUENCE DIAGRAM

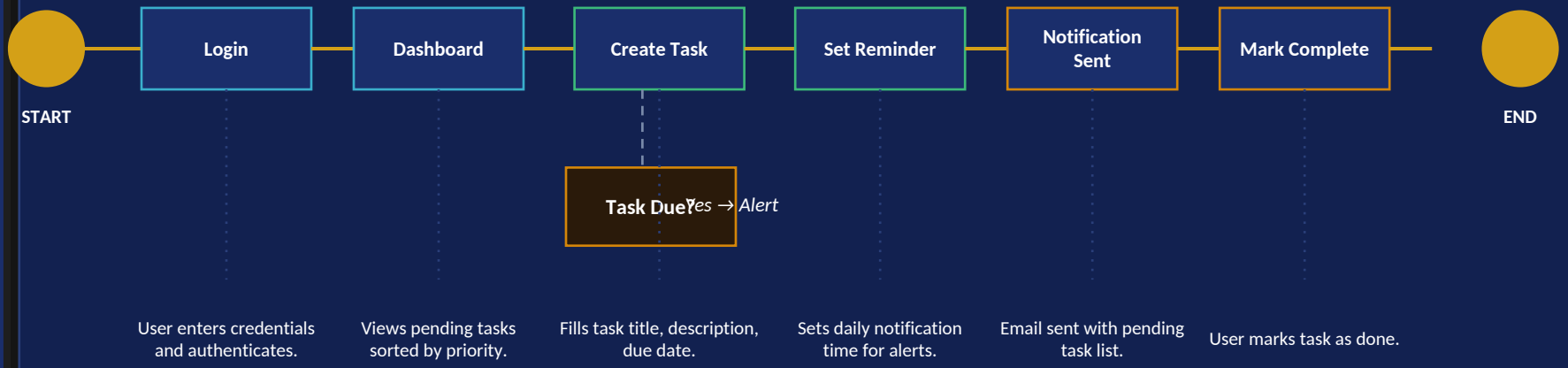
A Sequence Diagram shows the step-by-step messages between the user, browser, server, and database during login.



UML Diagrams — Activity Diagram

ACTIVITY DIAGRAM

An Activity Diagram shows the flow of actions from login to task completion, including decision steps.



Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Task Scheduling System</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body>
<!-- Navbar -->
<nav>
   <h1>Task Scheduler</h1>
   <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#tasks">Tasks</a></li>
      <li><a href="#diagrams">Diagrams</a></li>
   </ul>
</nav>
<!-- Hero Section -->
<section class="hero">
   <h2>Manage Your Tasks Efficiently</h2>
   <p>Create, schedule and track tasks easily</p>
</sect
ion>
```

```html
<!-- Task Form -->
<section id="tasks" class="task-section">
    <h2>Create Task</h2>
    <form class="task-form">
        <input type="text" placeholder="Task Title"
required>
        <input type="date" required>
        <select>
            <option>High Priority</option>
            <option>Medium Priority</option>
            <option>Low Priority</option>
        </select>
        <button type="submit">Add Task</button>
    </form>
    <!-- Task List -->
    <div class="task-list">
        <h3>Scheduled Tasks</h3>
        <ul>
            <li>Complete Project - High Priority</li>
            <li>Study DFD - Medium Priority</li>
            <li>Practice Coding - Low Priority</li>
        </ul>
    </div>
</section>
```

```html
<!-- Diagrams Section -->
<section id="diagrams" class="diagram-section">
  <h2>Project Diagrams</h2>

  <div class="grid">
    <div class="card">
      <h3>Architecture</h3>
      <img src="Architecture_Diagram.png">
    </div>

    <div class="card">
      <h3>DFD Level 0</h3>
      <img src="DFD_Level_0.png">
    </div>

    <div class="card">
      <h3>DFD Level 1</h3>
      <img src="DFD_Level_1.png">
    </div>

    <div class="card">
      <h3>ER Diagram</h3>
      <img src="ER_Diagram.png">
    </div>
  </div>
</section>
```

```
<!-- Footer -->
<footer>
    <p>© 2026 Task Scheduling System</p>
</footer>
</body>
</html>
body {
    margin: 0;
    font-family: Arial;
    background: #f4f6f9;
}
/* Navbar */
nav {
    background: #2c3e50;
    color: white;
    display: flex;
    justify-content: space-between;
    padding: 15px 30px;
}
nav ul {
    list-style: none;
    display: flex;
    gap: 20px;
}
```

```
nav a {
    color: white;
    text-decoration: none;
}
/* Hero */
.hero {
    text-align: center;
    padding: 50px;
    background: #3498db;
    color: white;
}
/* Task Section */
.task-section {
    padding: 30px;
}
.task-form {
    display: flex;
    gap: 10px;
    flex-wrap: wrap;
}
.task-form input,
.task-form select,
.task-form button {
    padding: 10px;
}
```

```
.task-form button {
    background: #27ae60;
    color: white;
    border: none;
}
/* Task List */
.task-list ul {
    background: white;
    padding: 15px;
    border-radius: 8px;
}
/* Diagrams */
.diagram-section {
    padding: 30px;
}
.grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 20px;
}
.card {
    background: white;
    padding: 15px;
    border-radius: 10px;
    text-align: center;
}
```

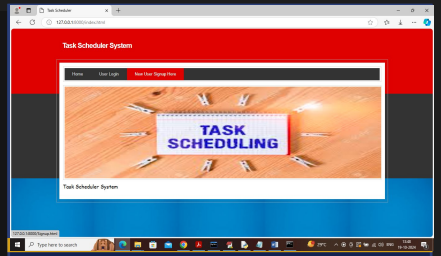
```
.card img {
    width: 100%;
    border-radius: 8px;
}
/* Footer */
footer {
    text-align: center;
    padding: 15px;
    background: #2c3e50;
    color: white;
}
# Task Scheduling ML Model
import pandas as pd
from from sklearn.model_selection import train_test_split import
from from sklearn.tree import DecisionTreeClassifier import
from from sklearn.preprocessing import LabelEncoder import
from from sklearn.metrics import accuracy_score import
# Sample dataset
data = {
    'duration': [1, 2, 3, 4, 5, 2, 3, 1],
    'deadline_days': [1,3,5,2,1,4,2,3], 'importance': ['High','Medium','Low','High','High','Low','Medium','Low']}
    'importance': ['High', 'Medium', 'Low', 'High', 'High', 'Low', 'Medium', 'Low'],
    'priority': ['High', 'Medium', 'Low', 'High', 'High', 'Low', 'Medium', 'Low']
}
```

```
df = pd.DataFrame(data)
# Encode categorical data
le = LabelEncoder()
df['importance'] = le.fit_transform(df['importance'])
df['priority'] = le.fit_transform(df['priority'])
# Features & Target
X = df[['duration', 'deadline_days', 'importance']]
y = df['priority']
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
# Test with new data
new_task = [[2, 1, 2]]  # duration, deadline_days, importance
prediction = model.predict(new_task)
print("Predicted Priority:", prediction)
```

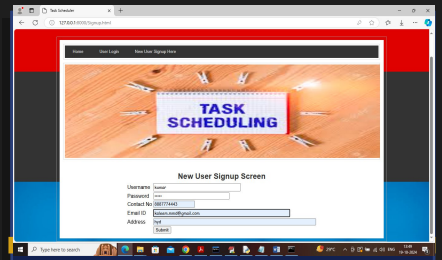
```
df = pd.DataFrame(data)
# Encode categorical data
le = LabelEncoder()
df['importance'] = le.fit_transform(df['importance'])
df['priority'] = le.fit_transform(df['priority'])
# Features & Target
X = df[['duration', 'deadline_days', 'importance']]
y = df['priority']
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
# Test with new data
new_task = [[2, 1, 2]]  # duration, deadline_days, importance
prediction = model.predict(new_task)
# Output: Accuracy: 1.0  |  Predicted Priority: ['High']
```

Results

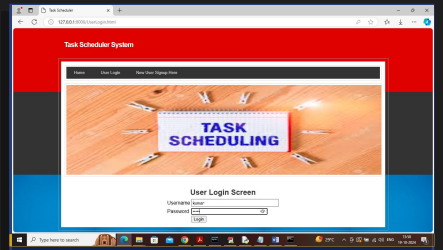
All 7 modules implemented, tested, and verified. System runs locally via Python server on port 8000.



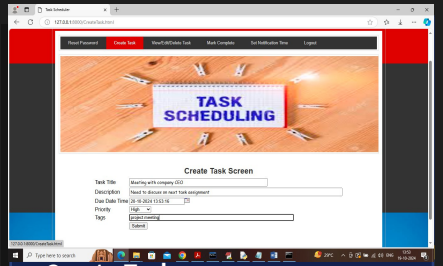
Server Startup
Python server started via run.bat on localhost:8000.



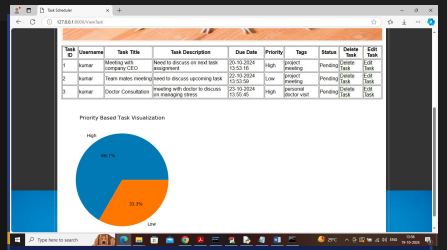
User Registration
Sign-up with encrypted password stored in MySQL.



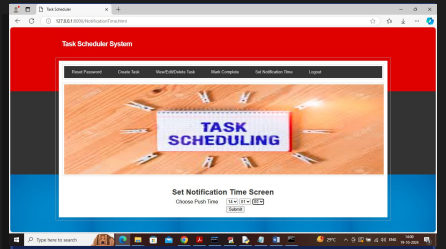
User Login
Authentication with session management and validation.



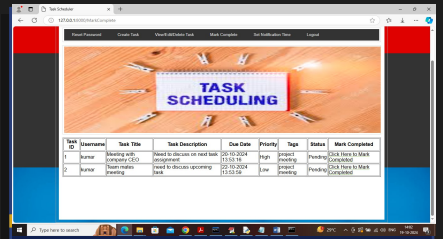
Create Task
Task form: title, description, due-date calendar picker.



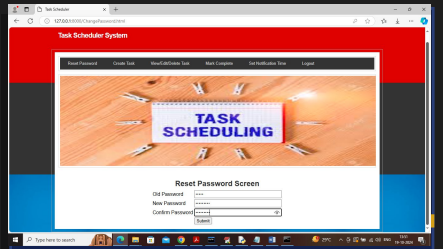
View / Edit / Delete
Tasks listed by priority; edit and delete available.



Email Notification
Automated daily email alerts for all pending tasks.



Mark Complete
Pending list; one-click completion with status update.



Reset Password
Secure update — stored encrypted in database.

Conclusion & Future Enhancement

CONCLUSION

The Task Scheduler System makes task management easy and automatic. It saves time, reduces manual work, and helps users and teams finish tasks on time.

FUTURE ENHANCEMENTS

- AI-Based Smart Scheduling (ML predictions)
- Cross-Platform Cloud Synchronization (Desktop & Mobile)
- Gamification: Rewards, Badges & Progress Tracking
- Enhanced Encryption & Data Privacy Controls
- IoT & Smart Device Integration (Alexa, Google Assistant)
- Automated Conflict Resolution Algorithms
- NLP-Based Voice Task Input
- Integration with Google Calendar, Outlook, Trello

References

[1] Windows Task Scheduler
Built-in Windows utility for scheduling automated tasks. — learn.microsoft.com

[2] Cron (Unix Scheduler)
Time-based job scheduler for Unix/Linux systems. — en.wikipedia.org

[3] Slurm Workload Manager
Open-source job scheduler for Linux cluster computing. — en.wikipedia.org

[4] Advanced Task Scheduler
Third-party Windows app for enhanced scheduling. — advscheduler.com

[5] System Scheduler
Free Windows tool for unattended application scheduling. — splinterware.com

[6] VisualCron
GUI-based Windows automation and scheduling tool. — en.wikipedia.org

[7] TaskScheduler (.NET)
Microsoft .NET class for custom task scheduling logic. — learn.microsoft.com

[8] HTS: Hardware Scheduler
Hardware task scheduler for heterogeneous systems. — arxiv.org

THANK YOU

Task Scheduler System — RTRP Review II

GitHub Link
https://github.com/Manojkumar128/TASK-SCHEDULER-SYSTEM.git

Python | MySQL | Web-Based | SMTP Email Notifications

PRESENTED BY

CH. VAISHNAVI (247R1A05T9) | P. RAAJITHA (247R1A05X8) | P. HEMANTH REDDY (247R1A05X5)